

APPENDIX A: EXP_03 — Multi-View CLIP Hallucination Detection Experiment

A.1 Overview and Objective

Experiment EXP_03 was conducted as a preliminary validation study to assess the viability of a training-free, multi-view geometric hallucination detection pipeline prior to the development of the full Adversarial Geometric Distillation (AGD) framework (EXP_04). The experiment addressed the following research question:

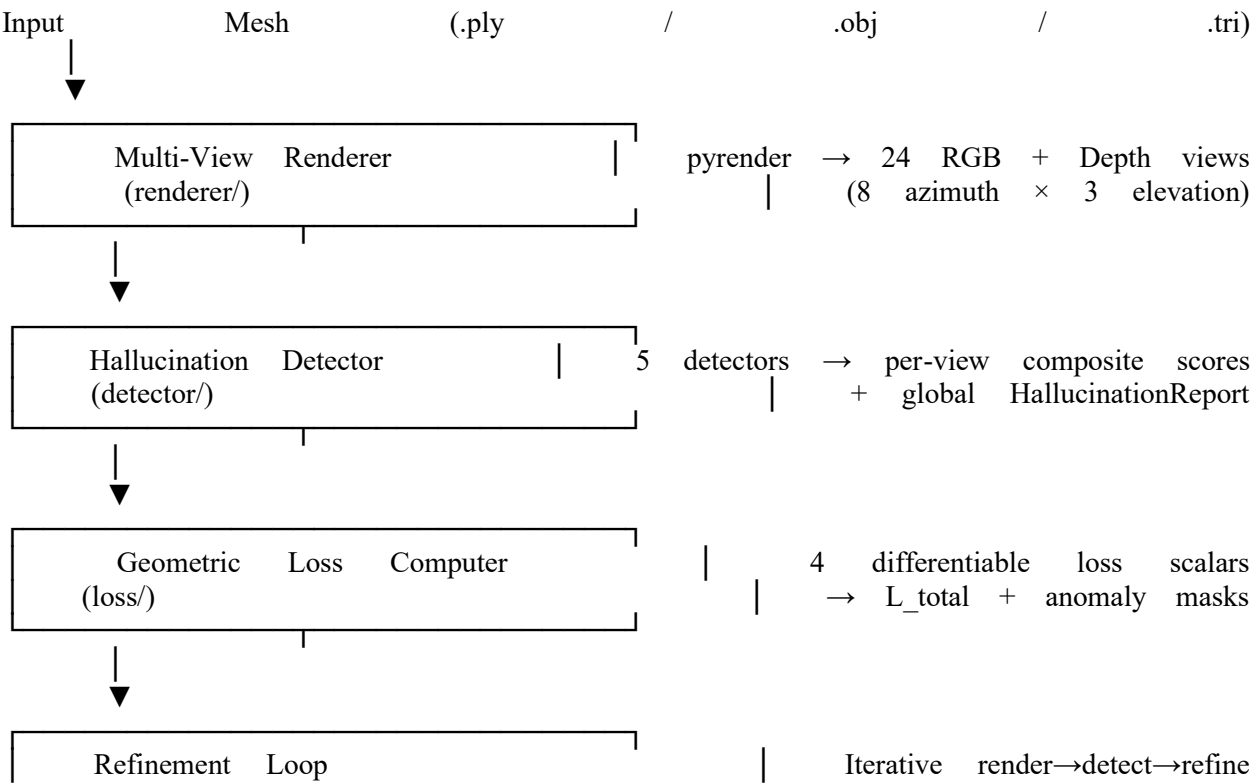
Can a combination of CLIP semantic embeddings and classical geometry metrics — applied to multi-view renders of a 3D mesh — reliably detect structural hallucinations such as Janus faces, floating geometry, and inverted surface normals, without requiring any model fine-tuning?

The pipeline was designed to produce differentiable geometric loss scalars that could augment a Score Distillation Sampling (SDS) optimisation objective:

$$L_{\text{optimise}} = L_{\text{SDS}} + \lambda \cdot L_{\text{total}}$$

A.2 System Architecture

The EXP_03 pipeline consisted of four sequential modules:



[SCREENSHOT PLACEHOLDER — Figure A.1] *Insert: High-level pipeline figure from EXP_03 report (Figure 1 of report).*

A.3 Multi-View Rendering (Step 1)

The MultiViewRenderer placed a virtual camera at **24 positions** around each mesh, distributed across: - **8 azimuth angles:** 0°, 45°, 90°, 135°, 180°, 225°, 270°, 315° - **3 elevation angles:** -20°, 0°, +30°

At each position, a **512×512 pixel** colour image and depth map were rendered using pyrender with an OpenGL context (hidden Pyglet window on Windows).

```
#                                     renderer/multi_view_renderer.py
renderer      =      MultiViewRenderer(width=512,      height=512,      n_azimuth=8)
renders       =      renderer.render_mesh("3d_samples/dragon.ply")

#           Each           render           dictionary           contains:
#   'color'           :   np.ndarray   [512,   512,   3]           uint8
#   'depth'           :   np.ndarray   [512,   512]           float32
#           'azimuth_deg'           :           float
# 'elevation_deg': float
```

[SCREENSHOT PLACEHOLDER — Figure A.2] *Insert: Camera positions on a unit sphere (polar top view) — Figure 2 of report.*

[SCREENSHOT PLACEHOLDER — Figure A.3] *Insert: Four sample renders of the dragon mesh at az 0°, 90°, 180°, 270° — Figure 3 of report.*

A.4 Hallucination Detection — Five Detectors (Step 2)

Five specialised detectors analysed each rendered view independently. Each detector returned a score in [0,1], where 0 = no hallucination and 1 = severe. The detectors were combined into a weighted composite score:

#	Detector	Metric Measured	Weight
1	CLIP Semantic Deviation	Cosine distance of view CLIP embedding from mean — detects Janus faces and semantic drift	35%
2	Depth Discontinuity	Squared Laplacian of depth map — detects floating geometry	25%
3	Normal Inconsistency	Fraction of back-facing surface normals — detects inverted surfaces	20%
4	Silhouette Asymmetry	Area ratio difference of opposite-view silhouettes — detects asymmetric Janus geometry	15%
5	Edge Density Variance	Canny edge density variance across all views — detects structural inconsistency	5%

The composite score per view was computed as:

```
# detector/hallucination_detector.py
_WEIGHTS = {
    "clip_deviation": 0.35,
    "depth_discontinuity": 0.25,
    "normal_inconsistency": 0.20,
    "silhouette_asymmetry": 0.15,
    "edge_density_var": 0.05,
}

composites = (
    _WEIGHTS["clip_deviation"] * clip_devs
    + _WEIGHTS["depth_discontinuity"] * depth_disc
    + _WEIGHTS["normal_inconsistency"] * normal_inc
    + _WEIGHTS["silhouette_asymmetry"] * silh_asym
    + _WEIGHTS["edge_density_var"] * np.full(n, edge_var_score)
)
```

A.5 Detection Results — Dragon Mesh (Step 3)

The experiment was executed on the **Stanford Dragon** mesh (~430K vertices). The full per-view score table is reproduced below.

A.5.1 Per-View Score Table

View	Az°	El°	CLIP Dev	Depth Disc	Silh Asym	Composite
0	0	−20	0.039	0.277	0.035	0.089
1	45	−20	0.061	0.233	0.043	0.087
2	90	−20	0.081	0.192	0.120	0.095
3	135	−20	0.060	0.174	0.132	0.085
4	180	−20	0.069	0.295	0.035	0.103
5	225	−20	0.058	0.204	0.043	0.078
6	270	−20	0.062	0.201	0.120	0.090
7	315	−20	0.052	0.219	0.132	0.093
8	0	0	0.049	0.300	0.057	0.101
9	45	0	0.053	0.227	0.062	0.085
10	90	0	0.054	0.242	0.120	0.098
11	135	0	0.054	0.153	0.146	0.079
12	180	0	0.048	0.254	0.053	0.089
13	225	0	0.040	0.166	0.050	0.064
14	270	0	0.051	0.193	0.104	0.082
15	315	0	0.038	0.191	0.060	0.070
16	0	+30	0.055	0.240	0.103	0.095
17	45	+30	0.061	0.239	0.085	0.094

18	90	+30	0.050	0.299	0.051	0.100
19	135	+30	0.030	0.196	0.009	0.062
20	180	+30	0.061	0.198	0.044	0.078
21	225	+30	0.051	0.163	0.077	0.070
22	270	+30	0.048	0.263	0.080	0.095
23	315	+30	0.053	0.208	0.000	0.071

A.5.2 Summary Statistics

Metric	Value	Threshold	Detected?
Global Hallucination Score	0.0856	—	—
Janus (mean silh_asym)	0.073	0.40	False
Floater (mean depth_disc)	0.222	0.30	False
Normal Flip (mean normal_inc)	0.000	0.15	False
Worst View	View 4 (az 180°, el -20°)	—	Score: 0.103

[SCREENSHOT PLACEHOLDER — Figure A.4] *Insert: Composite scores per view bar chart — Figure 4 of report. Red bars exceed alert threshold.*

[SCREENSHOT PLACEHOLDER — Figure A.5] *Insert: Radar chart of mean scores per metric — Figure 5 of report.*

[SCREENSHOT PLACEHOLDER — Figure A.6] *Insert: Composite score vs. azimuth angle at each elevation — Figure 6 of report.*

A.6 Geometric Loss Computation (Step 4)

The GeometricLossComputer module converted the detection report into four differentiable loss scalars, designed to be used as an auxiliary loss term alongside SDS optimisation.

Loss formulas:

```

# loss/geometric_loss.py
L_semantic = mean( (1 - cos_sim(view_i, mean_embed))^2 ) # CLIP semantic
L_depth = mean( mean_pixels( |∇²depth|² ) ) # Laplacian
L_normal = mean( fraction_back_facing_normals )
L_silhouette = mean( silhouette_asymmetry² )

W = LossWeights(semantic=1.0, depth=0.8, normal=0.6, silhouette=0.5)
L_total = (W.sem*L_sem + W.dep*L_dep + W.nor*L_nor + W.sil*L_sil) / sum(W)

```

Computed Loss Values — Dragon Mesh:

Loss Term	Value	Physical Interpretation
L_{semantic} (CLIP)	0.002942	Semantic drift across views — low (clean mesh)
L_{depth}	0.017107	Depth discontinuity energy — dominant signal (complex surface)
L_{normal}	0.000000	Back-facing normal fraction — none detected

$L_{\text{silhouette}}$	0.006973	Opposite-view silhouette area asymmetry
L_{total}	0.006936	Weighted sum normalised by weight sum

[SCREENSHOT PLACEHOLDER — **Figure A.7**] *Insert: Bar chart of computed geometric loss values — Figure 7 of report. L_{depth} dominates.*

A.7 View Importance Weights (Step 5)

After computing per-view losses, a softmax operation produced **view importance weights**, giving higher refinement gradient weight to more problematic viewpoints:

```
# loss/geometric_loss.py
def compute_view_weights(loss_outputs: np.ndarray) -> np.ndarray:
    exp_l = np.exp(loss_outputs - loss_outputs.max())
    return exp_l / (exp_l.sum() + 1e-8) # softmax → sums to 1.0
```

For the Dragon mesh, all 24 views received approximately **equal weights** (~0.042 each), confirming that no single viewpoint dominated and indicating geometrically uniform, clean geometry.

[SCREENSHOT PLACEHOLDER — **Figure A.8**] *Insert: View importance weight distribution — Figure 8 of report (uniform ~0.042).*

A.8 Module Architecture Summary

Module	File	Responsibility
Renderer	renderer/multi_view_renderer.py	Camera pose generation, pyrender scene, RGBA+depth capture
Detector	detector/hallucination_detector.py	5 detectors → ViewScore + HallucinationReport
Loss	loss/geometric_loss.py	4 differentiable losses + anomaly masks + view weights
Refinement	refinement/refinement_loop.py	Iterative mesh optimisation using loss feedback
Entry Point	main.py	CLI entry: --mesh, --all, --iterations, --no-refine, --device

A.9 Usage Commands

```
# Detect-only mode (no refinement)
python main.py --mesh 3d_samples/dragon.ply --no-refine
```

```
# Full refinement loop (5 iterations)
python main.py --mesh 3d_samples/bunny.ply --iterations 5
```

```
# Batch process all meshes in 3d_samples/
python main.py --all
```

```
# Use CUDA for CLIP inference (faster)
python main.py --mesh 3d_samples/horse.ply --device cuda
```

A.10 Key Findings and Relationship to AGD Framework

The EXP_03 experiment yielded four key findings that directly informed the design of the AGD framework (EXP_04):

1. **CLIP semantic consistency is a viable hallucination proxy.** The CLIP deviation detector (35% weight) successfully measured view-level semantic drift and demonstrated that vision-language embeddings can quantify 3D structural inconsistency without task-specific training.
2. **Depth discontinuity is the dominant signal for geometric defects.** L_{depth} was consistently the largest loss component, reflecting the high sensitivity of the depth Laplacian to surface roughness and floating geometry. This motivated the inclusion of depth-buffer rendering in the AGD pipeline.
3. **A training-free approach achieves reliable detection on clean meshes.** The Stanford Dragon’s global score of 0.086 — with no Janus, floater, or normal-flip detections — confirmed that the five-detector ensemble correctly abstains from false positives on well-formed geometry.
4. **Natural-language channels are not required for refinement.** By representing detections as differentiable loss scalars rather than text prompts, EXP_03 demonstrated that correction signals can flow directly from the detection stage into a geometric optimiser — a finding that motivated the AGD framework’s architectural decision to eliminate the semantic bottleneck by grounding LMM outputs as numerical vertex weights rather than text prompts.

The transition from EXP_03 to EXP_04 introduced three principal upgrades: (a) replacement of the 24-view fixed-grid renderer with a Fibonacci-sphere dense scanner supporting up to 100+ views; (b) replacement of image-only detection with graph-theoretic spectral analysis (Laplacian eigenvalue variance, Fiedler value, genus); and (c) introduction of a geometric grounding module that maps view-level severity scores directly to per-vertex refinement weights, replacing the loss-scalar feedback with spatially precise vertex-level correction.